


Universidad Autónoma de Chihuahua

Facultad de Ingeniería

Ingeniería en Computación

# Proyecto 2a Evaluación

## MongoDB: películas en clúster distribuido

Bases de Datos Avanzadas

Docente: JOSE SAUL DELIRA MIRAMONTES

**Carlos González Carrera**
Matrícula: 385574

**Elio Castillo Reyes**
Matrícula: 385770

**Jose Fernando Martinez Pacheco**
Matrícula: 385795

4 de noviembre de 2026

# 1. Introducción y objetivo

Esta práctica retoma el clúster de MongoDB de la Actividad 1 y lo utiliza con la base sample_mflix y la colección movies. El objetivo es agregar una clave de particionamiento hashed, trabajar con películas mediante operaciones CRUD y revisar cómo se reparten los documentos entre shards.

## 2. Componentes utilizados

| Servicio | Función | Puerto del contenedor |
|---|---|---|
| configsvr | Metadatos de configuración | 27019 |
| shard1 | Primer fragmento de datos | 27018 |
| shard2 | Segundo fragmento de datos | 27018 |
| shard3 | Tercer fragmento de datos | 27018 |
| mongos | Router de consultas | 27017 |

Docker Compose define cinco contenedores. El puerto local 27020 se dirige al router mongos. Los archivos 01-configsvr.js y 02-shard*.js inicializan el conjunto de configuración y los conjuntos de los tres shards; 03-configurar-cluster.js registra los shards y habilita el particionamiento.

## Secuencia de preparación

```
docker compose up -d
docker exec practica-configsvr mongosh --port 27019 /scripts/01-configsvr.js
docker exec practica-shard1 mongosh --port 27018 /scripts/02-shard1.js
docker exec practica-shard2 mongosh --port 27018 /scripts/02-shard2.js
docker exec practica-shard3 mongosh --port 27018 /scripts/02-shard3.js
docker exec practica-mongos mongosh /scripts/03-configurar-cluster.js
```

> Los cuatro replica sets configurados aquí tienen un solo miembro cada uno. El proyecto no incorpora nodos secundarios de réplica.

## 3. Base de datos, colección y shard key

El archivo scripts/06-configurar-movies.js prepara sample_mflix.movies y aplica la clave id_movie con estrategia hashed. Cada documento queda asociado a una partición según el hash de ese identificador.

```
const database = db.getSiblingDB("sample_mflix");
database.createCollection("movies");
sh.shardCollection("sample_mflix.movies", { id_movie: "hashed" });
database.movies.getShardVersion();
sh.status();
```

## 4. Carga de películas

app/importar_peliculas.py se conecta a localhost:27020, lee un archivo JSON con películas, asigna un id_movie consecutivo desde 1 y las inserta en lotes de 500. Antes de importar revisa si la colección está vacía para evitar repetir la carga.

```
ARCHIVO = "data/sample_mflix.movies.json"
for indice, pelicula in enumerate(peliculas, start=1):
    pelicula["id_movie"] = indice
for inicio in range(0, len(peliculas), 500):
    coleccion.insert_many(peliculas[inicio:inicio + 500],
                          ordered=False)
```

### Archivo necesario

La carpeta data/ está excluida del repositorio. Para repetir la práctica, es necesario colocar el archivo sample_mflix.movies.json dentro de esa carpeta antes de ejecutar python app/importar_peliculas.py. El reporte no presupone una cantidad concreta de películas importadas.

## 5. CRUD y seguimiento del reparto

app/crud_peliculas.py presenta un menú con cuatro operaciones sobre sample_mflix.movies. Usa id_movie para identificar la película cuando se modifica o elimina un documento.

| Opción | Función | Operación |
|---|---|---|
| 1 | insertar_pelicula() | insert_one |
| 2 | consultar_peliculas() | find y sort |
| 3 | actualizar_pelicula() | update_one con $set |
| 4 | eliminar_pelicula() | delete_one |

Después de cada opción, mostrar_distribucion() consulta el estado de la colección fragmentada. La agregación se ejecuta sobre admin y lee numOwnedDocuments para cada shard.

```
 resultado = cliente.admin.aggregate([
     {"$shardedDataDistribution": {}},
    {"$match": {"ns": "sample_mflix.movies"}}
])
for documento in resultado:
    for shard in documento["shards"]:
        print(shard["shardName"], shard["numOwnedDocuments"])
```

## 6. Ejemplo de distribución entre shards

app/carga_distribucion.py consulta la distribución inicial, prepara 300 películas de prueba con id_movie de 21350 a 21649 y vuelve a consultar los documentos de cada shard. Si esos IDs ya existen, el programa cancela la inserción para no duplicarlos.

```
print("Distribucion inicial")
mostrar_distribucion()
existentes = coleccion.count_documents({
    "id_movie": {"$gte": 21350, "$lte": 21649}
})
if existentes == 0:
    resultado = coleccion.insert_many(nuevas)
    mostrar_distribucion()
    print("Total:", coleccion.count_documents({}))
```

### Cómo se comprueba

La salida de mostrar_distribucion() permite comparar el conteo de documentos en cada shard antes y después de la carga. El uso de una clave hashed busca repartir valores consecutivos entre distintas particiones, aunque no garantiza por sí mismo tres conteos idénticos.

## 7. Resultado del desarrollo

El repositorio reúne Docker Compose, los scripts de configuración, los programas de Python y el archivo de dependencias. El programa de carga asigna id_movie a las películas; el menú permite insertarlas, consultarlas, actualizarlas y eliminarlas. Las funciones de monitoreo muestran la distribución por shard durante estas operaciones.

### Conclusión

El segundo proyecto extiende el clúster de la Actividad 1 con una colección de películas y una clave hashed. Con los programas se pueden importar los datos, realizar operaciones CRUD y comparar cómo se distribuyen los documentos entre los tres shards.

### Fuentes

Código de los repositorios de Actividad 1 y Proyecto Segundo Parcial. MongoDB Manual: Hashed Sharding; $shardedDataDistribution; Replica Set Deployment Architectures (mongodb.com/docs/manual/).